راهنمای تحویل چالش‌های ترم سوم

این فایل برای هر چهار چالش معتبر است. قبل از شروع هر مرحله یک‌بار بخوانید؛ شرایط تحویل در همه مراحل یکسان است و در فایل هر چالش تکرار نمی‌شود.

---

۱. ساختار کلی مسیر

چالش‌ها پشت سر هم هستند، نه موازی:

مرحله	فایل	موضوع اصلی	خروجی‌اش ورودی چه مرحله‌ای است؟
---	---	---	---
۱	Challenge_Data_Factory_01	مهندسی Python، API، Pipeline، Testing، Git	مرحله ۲ و چالش نهایی
۲	Challenge_Database_Performance_02	PostgreSQL، Index، Execution Plan، View، Procedure	مرحله ۳ و چالش نهایی
۳	Challenge_Machine_Learning_03	آماده‌سازی داده، Feature Engineering، Classification، Evaluation	چالش نهایی
نهایی	Final_Challenge_Capstone_04	یکپارچه‌سازی هر سه مرحله در یک سامانه	—

قاعده: کد مرحله قبل دور ریخته نمی‌شود. در هر مرحله باید از همان package و همان Database که خودتان ساخته‌اید استفاده کنید. اگر مرحله ۱ بد نوشته شده باشد، مرحله ۲ و ۳ گران تمام می‌شود — این بخشی از طراحی چالش است.

---

۲. قالب پاسخ: پاسخ باید در قالب Notebook باشد

پاسخ اصلی هر مرحله یک Jupyter Notebook است (solution.ipynb). شرایط پذیرش:

1. اجرای کامل از صفر: با Restart Kernel & Run All روی یک kernel تازه باید کامل و بدون خطا اجرا شود. هیچ سلولی نباید به حالتی وابسته باشد که دستی ساخته شده است.
2. شماره اجرای سلول‌ها باید پیوسته و از ۱ تا N باشد. سلولی با شماره پراکنده یا خالی = اجرانشده.

3. خروجی‌ها باید ذخیره‌شده باشند. نوتبوک با خروجی پاک‌شده تحویل نمی‌شود؛ ما باید بدون اجرای مجدد هم بتوانیم نتایج را ببینیم.

4. مسیرها نسبی باشند (.../data/raw)؛ هیچ مسیر مطلقی از ماشین خودتان (C:\Users\...) نباید وجود داشته باشد.

5. Seed تصادفی ثابت باشد و در ابتدای نوتبوک یک‌جا تعریف شده باشد. دو اجرای متوالی باید همان اعداد را بدهد.

6. منطق سنگین داخل نوتبوک نوشته نمی‌شود. توابع و کلاس‌ها در فایل‌های .py پروژه زندگی می‌کنند و نوتبوک آن‌ها را import می‌کند و روایت می‌کند. نوتبوکی که ۴۰۰ خط کد خطی باشد نمره‌اش کم می‌شود.

7. روایت داشته باشد: هر بخش با یک سلول Markdown شروع شود که می‌گوید در این بخش چه چیزی و چرا بررسی می‌شود. نوتبوک باید برای یک خوانندهٔ ناآشنا قابل دنبال کردن باشد.

8. سقف زمان اجرا در هر مرحله اعلام شده است. اگر از آن رد شد، باید در گزارش دلیلش را بنویسید.

9. وابستگی‌ها قفل‌شده باشند: یک requirements.txt با نسخه دقیق — مثلاً pandas==۲.۲.۳، نه pandas و نه pandas>=۲.

زبان سلول‌های Markdown فارسی است؛ نام متغیرها، توابع، commit message و docstring انگلیسی.

---

۳. فایل گزارش: «دلیل حل»

در کنار نوتبوک یک فایل report.md (یا report.docx) تحویل می‌دهید. این فایل شرح کد نیست؛ شرح *تصمیم* است. اگر جمله‌ای از آن را بتوان با خواندن کد فهمید، آن جمله جای دیگری اشتباه نشسته است.

گزارش باید این بخش‌ها را داشته باشد:

1. فهم مسئله: مسئله را با زبان خودتان بازنویسی کنید. چه چیزی را فرض گرفتید که در صورت مسئله نیامده بود؟

2. تصمیم‌های کلیدی: حداقل ۵ تصمیم را فهرست کنید. برای هرکدام:
   - گزینه‌هایی که داشتید،
   - گزینه‌ای که انتخاب کردید،
   - شاهدِ عددی یا منطقی انتخاب (زمان اجرا، تعداد رکورد، مقدار متریک، پلن اجرا...)،
   - چه چیزی را با این انتخاب از دست دادید.

3. چیزهایی که شکست خوردند: حداقل یک مسیر که امتحان کردید و کنار گذاشتید، با دلیل. گزارشی که همه‌چیزش بار اول درست شده باشد باورپذیر نیست.

4. محدودیت‌ها و ریسک‌ها: کجای کار شما اگر داده عوض شود می‌شکند؟

5. اگر دو روز دیگر وقت داشتید چه می‌کردید؟ (حداکثر یک پاراگراف)

قاعده سخت: هر عددی که در گزارش می‌نویسید باید در نوتبوک قابل ردیابی باشد. عدد بدون منبع، غلط حساب می‌شود.

طول پیشنهادی: ۸۰۰ تا ۱۵۰۰ کلمه. گزارش بلندتر از این خوانده می‌شود اما امتیاز اضافه ندارد.

۴. ساختار پوشه تحویل

```
stage-<n>-<your-name>/
├── solution.ipynb          # پاسخ اصلی
├── report.md               # دلیل حل
├── requirements.txt
├── README.md               # چطور اجرا کنیم؟ (۱۰ خط کافی است)
├── src/                    # کد واقعی پروژه
├── tests/
├── data/
│   ├── raw/                # داده خام — در Git نمی‌رود
│   └── curated/            # خروجی‌ها
└── .gitignore
```

کل پوشه را به‌صورت یک repository گیت (با history واقعی) تحویل دهید، نه یک ZIP از فایل‌های نهایی.

---

۵. چه چیزی به شما گفته نمی‌شود

این بخش را جدی بگیرید، چون ساختار چالش روی آن بنا شده است:

- **راه حل گفته نمی‌شود. نه در صورت مسئله، نه در پاسخ به سؤال‌ها.**
- **مسیر حل هم گفته نمی‌شود. به شما نمی‌گوییم با چه تابعی، چه الگویی یا به چه ترتیبی کار را انجام دهید. انتخاب معماری، انتخاب کتابخانه در میان ابزارهای مجاز، و ترتیب کار بخشی از مسئله است.**
- **کد آغازین (starter code) داده نمی‌شود.**
- **عدد هدف برای «درست» بودن داده نمی‌شود، مگر جایی که صریحاً به‌عنوان بودجه یا آستانه پذیرش نوشته شده باشد.**

سؤال پرسیدن آزاد است، اما فقط درباره صورت مسئله: «منظورتان از فلان فیلد چیست؟»، «آیا این رکورد معتبر حساب می‌شود؟»، «بودجه زمانی شامل زمان دانلود هم هست؟». سؤال‌هایی مثل «از چه مدلی استفاده کنم؟» یا «کدام Index را بسازم؟» پاسخ داده نمی‌شوند — آن سؤال، خودِ مسئله است.

---

۶. ابزارهای مجاز

دامنه ابزارها همان سرفصل سطح ۳ است:

- Python (+ venv/pip), ماژول و package، unittest
- Git
- pandas, requests, json
- PostgreSQL, SQLAlchemy
- scikit-learn و یک کتابخانه نمودار به انتخاب خودتان

استفاده از ابزاری بیرون این دامنه (مثلاً یک framework آماده orchestration یا AutoML) تنها در صورتی قبول است که در گزارش توضیح دهید چرا و چه چیزی را جایگزین کرده است. ابزاری که کار اصلیِ مورد سنجش را به‌جای شما انجام دهد، امتیاز آن بخش را صفر می‌کند.

---

۷. سیاست استفاده از دستیار هوش مصنوعی

استفاده مجاز است و پنهان کردنش مجاز نیست. در report.md یک بخش کوتاه به نام «AI usage» اضافه کنید: کجا کمک گرفتید و چه چیزی را خودتان تغییر دادید. در جلسه دفاع، از هر خطی از کدتان ممکن است پرسیده شود «چرا این‌طور؟» — پاسخ «نمی‌دانم» برای کد خودتان، همان اثر نداشتن کد را دارد.

---

۸. معیار کلی ارزیابی (مشترک همه مراحل)

محور	وزن
---	---
درستی نتیجه (آیا مسئله واقعاً حل شده؟)	٪۳۰
کیفیت مهندسی (ساختار، خوانایی، تست، Git)	٪۲۵
شاهد و استدلال (گزارش، اعداد، پلن‌ها، مقایسه‌ها)	٪۲۵
قابلیت بازتولید (اجرای کامل روی ماشین ما)	٪۲۰

یک نکته: اگر پروژه روی ماشین ما اجرا نشود، سایر محورها ارزیابی نمی‌شوند. قابلیت بازتولید پیش‌شرط است، نه یک امتیاز جانبی.